

PRACTICAL NO. 3

NAME : BHOSALE SHREYAS UDDHAV

PRN : 202501070125

BATCH : E21

3.1.1 Numpy Array Operations

Problem Statement:

Write a Python program to create a NumPy array based on user input and display the following:

- The created NumPy array.
- The number of dimensions of the array using ndim.
- The shape of the array using shape.
- The total number of elements in the array using size.

Assume all input elements to be valid integers.

Input Format:

- The first line contains two space-separated integers, R and C, representing the number of rows and the number of columns.
- The next R lines contain R space-separated integers representing the elements of the array, entered row by row.

Output Format:

- The created NumPy array (printed as a 2D array).
- An integer representing the number of dimensions of the array.
- A tuple representing the shape of the array.
- An integer representing the total number of elements in the array.

Note: Use reshape() function to reshape the input array with the specified number of rows and columns.

Code:

```
import numpy as np

rows_cols = list(int(input().split()))
elements = []
for _ in range(rows_cols[0]):
    elements.extend(map(int, input().split()))

arr2d = np.array(elements).reshape(rows_cols[0], rows_cols[1])

print(arr2d)
print(arr2d.ndim)
print(arr2d.shape)
print(arr2d.size)
```

Output:

Average time:
0.014 s
14.00 ms

Maximum time:
0.022 s
22.00 ms

2 out of 2 shown test case(s) passed

10 out of 10 hidden test case(s) passed

Test case 1

22 ms

Debug

Expected output

Actual output

3 4

3 4

1 2 3 4

1 2 3 4

5 6 7 8

5 6 7 8

9 10 11 12

9 10 11 12

[[1, 2, 3, 4]]

[[1, 2, 3, 4]]

[5, 6, 7, 8]

[5, 6, 7, 8]

[9, 10, 11, 12]]

[9, 10, 11, 12]]

2

2

(3, 4)

(3, 4)

12

12

Test case 2

7 ms

Debug

Expected output

Actual output

0 0

0 0

[]

[]

2

2

(0, 0)

(0, 0)

0

0

3.2.1 Numpy Matrix Operations

Problem Statement:

The program takes two 3x3 matrices, matrix_a and matrix_b, as input from the user and converts them into NumPy arrays.

Tasks: You are required to compute and display the results of the following matrix operations:

1. Addition (matrix_a + matrix_b)
2. Subtraction (matrix_a - matrix_b)
3. Element-wise Multiplication (matrix_a * matrix_b)
4. Matrix Multiplication (matrix_a dot matrix_b)
5. Transpose of Matrix A

Input Format:

- The user will input 3 rows for matrix_a, each containing 3 integers separated by spaces.
- The user will input 3 rows for matrix_b, each containing 3 integers separated by spaces.

Output Format: The program should display the results of the operations in the following order:

1. The result of Addition.
2. The result of Subtraction.
3. The result of Element-wise Multiplication.
4. The result of Matrix Multiplication.
5. The Transpose of Matrix A.

Code:

```
import numpy as np

# Input matrices
print('Enter Matrix A:')
matrix_a = np.array([[int(input().split()), input().split()[1],
input().split()[2]]])
matrix_a = np.array([[int(x) for x in input().split()] for _ in range(3)])

print('Enter Matrix B:')
matrix_b = np.array([[int(x) for x in input().split()] for _ in range(3)])

# Addition
print('Addition (A + B):')
print(matrix_a + matrix_b)

# Subtraction
print('Subtraction (A - B):')
print(matrix_a - matrix_b)

# Element-wise Multiplication
print('Element-wise Multiplication (A * B):')
print(matrix_a * matrix_b)
```

```
# Matrix multiplication (dot product)
print('A dot B:')
print(np.dot(matrix_a, matrix_b))

# Transpose
print('Transpose of A:')
print(matrix_a.T)
```

Output:

Average time:
0.045 s
45.50 ms

Maximum time:
0.052 s
52.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

Debug

Expected output

Actual output

Enter Matrix A:

Enter Matrix A:

1 2 3

1 2 3

4 5 6

4 5 6

7 8 9

7 8 9

Enter Matrix B:

Enter Matrix B:

1 1 1

1 1 1

1 1 1

1 1 1

1 1 1

1 1 1

Addition (A + B):

Addition (A + B):

[[2 3 4]

[[2 3 4]

[5 6 7]

[5 6 7]

[8 9 10]]

[8 9 10]]

Subtraction (A - B):

Subtraction (A - B):

[[0 1 2]

[[0 1 2]

[3 4 5]

[3 4 5]

[6 7 8]]

[6 7 8]]

Element-wise Multiplication (A * B):

Element-wise Multiplication (A * B):

[[1 2 3]

[[1 2 3]

[4 5 6]

[4 5 6]

[7 8 9]]

[7 8 9]]

A dot B:

A dot B:

[[6 6 6]

[[6 6 6]

[15 15 15]

[15 15 15]

[24 24 24]]

[24 24 24]]

Transpose of A:

Transpose of A:

[[1 4 7]

[[1 4 7]

[2 5 8]

[2 5 8]

[3 6 9]]

[3 6 9]]

Test case 2 44 ms Debug ≡ 🔍

Expected output

Enter Matrix A:

2 4 8

9 6 5

7 8 9

Enter Matrix B:

10 12 13

14 15 16

17 18 19

Addition (A + B):

[[12 16 21]

[23 21 21]

[24 26 28]]

Subtraction (A - B):

[[-8 -18 -5]

[-5 -9 -11]

[-10 -10 -10]]

Element-wise Multiplication (A * B):

[[20 48 104]

[126 90 80]

[119 144 171]]

A dot B:

[[212 228 242]

[259 288 308]

[335 366 390]]

Transpose of A:

[[2 9 7]

[4 6 8]

[8 5 9]]

Actual output

Enter Matrix A:

2 4 8

9 6 5

7 8 9

Enter Matrix B:

10 12 13

14 15 16

17 18 19

Addition (A + B):

[[12 16 21]

[23 21 21]

[24 26 28]]

Subtraction (A - B):

[[-8 -18 -5]

[-5 -9 -11]

[-10 -10 -10]]

Element-wise Multiplication (A * B):

[[20 48 104]

[126 90 80]

[119 144 171]]

A dot B:

[[212 228 242]

[259 288 308]

[335 366 390]]

Transpose of A:

[[2 9 7]

[4 6 8]

[8 5 9]]

3.2.2 Numpy: Horizontal and Vertical Stacking of Arrays

Problem Statement:

You are given two arrays arr1 and arr2. You need to perform horizontal and vertical stacking operations on them using NumPy.

- Horizontal Stacking: Stack the two matrices horizontally (side by side).
- Vertical Stacking: Stack the two matrices vertically (one below the other).

Input Format:

- The program should first prompt the user to input two 3x3 arrays.
- Each array consists of 3 rows, and each row contains 2 space-separated integers.
- The user will input the two arrays row by row.

Output Format:

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Code:

```
import numpy as np

# Input arrays
print('Enter Array1:')
arr1 = np.array([[int(x) for x in input().split()] for _ in range(3) if len(input())>0])
print('Enter Array2:')
arr2 = np.array([[int(x) for x in input().split()] for _ in range(3) if len(input())>0])

# Perform horizontal stacking (hstack)
hstack = np.hstack((arr1, arr2))
print('Horizontal Stack:')
print(hstack)

# Perform vertical stacking (vstack)
vstack = np.vstack((arr1, arr2))
print('Vertical Stack:')
print(vstack)
```

Output:

Average time: 0.049 s
Maximum time: 0.069 s
43.25 ms 63.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1

Expected output

Enter Array1:

1 2 3

4 5 6

7 8 9

Enter Array2:

4 5 6

7 8 9

10 11 12

Horizontal Stack:

[[1 2 3 4 5 6]

[4 5 6 7 8 9]

[7 8 9 10 11 12]]

Vertical Stack:

[[1 2 3]

[4 5 6]

[7 8 9]

[4 5 6]

[7 8 9]

[10 11 12]]

Actual output

Enter Array1:

1 2 3

4 5 6

7 8 9

Enter Array2:

4 5 6

7 8 9

10 11 12

Horizontal Stack:

[[1 2 3 4 5 6]

[4 5 6 7 8 9]

[7 8 9 10 11 12]]

Vertical Stack:

[[1 2 3]

[4 5 6]

[7 8 9]

[4 5 6]

[7 8 9]

[10 11 12]]

Test case 2

Expected output

Enter Array1:

-1 -2 -3

-4 -5 -6

-7 -8 -9

Enter Array2:

10 11 12

13 14 15

16 17 18

Horizontal Stack:

[[-1 -2 -3 10 11 12]

[-4 -5 -6 13 14 15]

[-7 -8 -9 16 17 18]]

Vertical Stack:

[[-1 -2 -3]

[-4 -5 -6]

[-7 -8 -9]

[10 11 12]

[13 14 15]

[16 17 18]]

Actual output

Enter Array1:

-1 -2 -3

-4 -5 -6

-7 -8 -9

Enter Array2:

10 11 12

13 14 15

16 17 18

Horizontal Stack:

[[-1 -2 -3 10 11 12]

[-4 -5 -6 13 14 15]

[-7 -8 -9 16 17 18]]

Vertical Stack:

[[-1 -2 -3]

[-4 -5 -6]

[-7 -8 -9]

[10 11 12]

[13 14 15]

[16 17 18]]

Terminal Test cases

Prev Reset Submit Next

3.2.3 Numpy: Custom Sequence Generation

Problem Statement:

Write a Python program that takes the following inputs from the user:

- Start value: The starting point of the sequence.
- Stop value: The sequence should end before this value.
- Step value: The increment between each number in the sequence.

The program should then generate a sequence using numpy based on these inputs and print the generated sequence.

Input Format:

- The user will input three integer values: start, stop, and step, each on a new line.

Output Format:

- The program should print the generated sequence based on the input values.

Code:

```
import numpy as np

# Take user input for the start, stop, and step of the sequence
start = int(input())
stop  = int(input())
step  = int(input())

# Generate the sequence using np.arange
a = np.arange(start, stop, step)
print(a)
# Print the generated sequence
```

Output:

Average time: **0.015 s**
15.50 ms

Maximum time: **0.020 s**
20.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 20 ms Debug 	
Expected output	Actual output
3	3
10	10
2	2
[3 5 7 9]	[3 5 7 9]

Test case 2 20 ms Debug 	
Expected output	Actual output
10	10
20	20
30	30
[10]	[10]

3.2.4 Numpy: Arithmetic and Statistical Operations, Mathematical Operations, Bitwise Operations

Problem Statement:

You are given two arrays A and B. Your task is to complete the function `array_operations(A, B)`, which will convert these lists into NumPy arrays and perform the following operations:

1. Arithmetic Operations:
 - Compute the element-wise sum, difference, and product of the two arrays.
2. Statistical Operations:
 - Compute the mean, median, and standard deviation of array A.
3. Bitwise Operations:
 - Perform Bitwise AND, Bitwise OR, and Bitwise XOR on the arrays (A, OR, A).

Input Format:

- The first line contains space-separated integers representing the elements of array A.
- The second line contains space-separated integers representing the elements of array B.

Output Format:

- For each operation (arithmetic, statistical, and bitwise), print the results in the specified format as shown in sample test cases.

Code:

```
import numpy as np

def array_operations(A, B):
    # Convert to numpy arrays
    a = np.array(A)
    b = np.array(B)
    ans_terms = B

    # Arithmetic Operations
    sum_result = a + b
    diff_result = a - b
    prod_result = a * b

    # Statistical Operations
    mean_A = np.mean(a)
    median_A = np.median(a)
    std_dev_A = np.std(a)

    # Bitwise Operations
    and_result = a & b
    or_result = a | b
    xor_result = a ^ b

    # Output results with one space between each element
    print('Element-wise Sum:', ' '.join(str(x) for x in sum_result))
```

```

    print('Element-wise Difference:', '-', '-'.join(str(abs(x)) for x in
diff_result))
    print('Element-wise Product:', ' '.join(str(x) for x in prod_result))

    print(f'Mean of A: {mean_A}')
    print(f'Median of A: {median_A}')
    print(f'Standard Deviation of A: {std_dev_A}')

    print(f'Bitwise AND: {and_result}')
    print(f'Bitwise OR: {or_result}')
    print(f'Bitwise XOR: {xor_result}')

A = [int(input().split()[0:3:1]) [i] for i in [0,1,2] if i < 3] # Elements of
array A
B = [int(input().split()[0:3:1]) [i] for i in [0,1,2] if i < 3] # Elements of
array B
array_operations(A, B)

```

Output:

Average time
0.016 s
16.00 ms

Maximum time

0.024 s

24.00 ms

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 25 ms Debug ⌵ ⌵

Expected output

1 2 3 4

5 6 7 8

Element-wise Sum: 6 8 10 12

Element-wise Difference: -4 -4 -4 -4

Element-wise Product: 5 12 21 32

Mean of A: 2.5

Median of A: 2.5

Standard Deviation of A: 1.118033988749895

Bitwise AND: 1 2 3 0

Bitwise OR: 5 6 7 12

Bitwise XOR: 4 4 4 12

Actual output

1 2 3 4

5 6 7 8

Element-wise Sum: 6 8 10 12

Element-wise Difference: -4 -4 -4 -4

Element-wise Product: 5 12 21 32

Mean of A: 2.5

Median of A: 2.5

Standard Deviation of A: 1.118033988749895

Bitwise AND: 1 2 3 0

Bitwise OR: 5 6 7 12

Bitwise XOR: 4 4 4 12

3.2.5. Numpy: Copying and Viewing Arrays

Problem Statement:

The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code for:

- Creating a view of the `original_array` and assigning it to `view_array`.
- Creating a copy of the `reshaped_array` and assigning it to `copy_array`.

After completing these steps, observe how modifying the view affects the `original_array`, while modifying the copy does not.

Input Format:

- A single line of space-separated integers.

Output Format:

- Original array after modifying view: `original_array`
- View array: `view_array`
- Original array after modifying copy: `original_array`
- Copy array: `copy_array`

Code:

```
import numpy as np

inspection = list(map(int, input().split(' ')))

# Original array
original_array = np.array(inspection)

# Create a view
view_array = original_array.view()

# Create a copy
copy_array = original_array.copy()

# Modify the view
view_array[0] = 99
print('Original array after modifying view:', original_array)
print('View array:', view_array)

# Modify the copy
copy_array[0] = 88
print('Original array after modifying copy:', original_array)
print('Copy array:', copy_array)
```

Output:

Average time
0.010 s
5.75 ms

Maximum time
0.014 s
14.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 14 ms Debug

Expected output
10 20 30 40 50 60 70 80
Original array after modifying view: [99 20 30 40 50 60 70 80]
View array: [99 20 30 40 50 60 70 80]
Original array after modifying copy: [99 20 30 40 50 60 70 80]
Copy array: [10 88 30 40 50 60 70 80]

Actual output
10 20 30 40 50 60 70 80
Original array after modifying view: [99 20 30 40 50 60 70 80]
View array: [99 20 30 40 50 60 70 80]
Original array after modifying copy: [99 20 30 40 50 60 70 80]
Copy array: [10 88 30 40 50 60 70 80]

Test case 2 10 ms Debug

Expected output
99 2 3 4 5
Original array after modifying view: [99 2 3 4 5]
View array: [99 2 3 4 5]
Original array after modifying copy: [99 2 3 4 5]
Copy array: [99 88 3 4 5]

Actual output
99 2 3 4 5
Original array after modifying view: [99 2 3 4 5]
View array: [99 2 3 4 5]
Original array after modifying copy: [99 2 3 4 5]
Copy array: [99 88 3 4 5]

3.2.6. Numpy: Searching, Sorting, Counting, Broadcasting

Problem Statement:

The given code in the editor takes a simple array, `arr_a`, as space-separated integers as input from the user. Additionally it takes the following input:

- `search_value`: The value to search for in the array.
- `count_value`: The value to count occurrences of in the array.
- `broadcast_value`: The value to add for broadcasting across the array.

You need to complete the code to perform the following operations:

1. Searching: Find the indices where `search_value` appears in `arr_a` and print these values.
2. Counting: Count how many times `count_value` appears in `arr_a` and print the count.
3. Broadcasting: Add `broadcast_value` to each element of `arr_a` using broadcasting, and print the resulting array.
4. Sorting: Sort `arr_a` in ascending order and print the sorted array.

Input Format:

1. A single line containing space-separated integers representing `arr_a`.
2. An integer `search_value` representing the value to search for in the array.
3. An integer `count_value` representing the value to count occurrences in the array.
4. An integer `broadcast_value` representing the value to add to each element of the array.

Output Format:

1. The indices where `search_value` occurs in `arr_a`.
2. The count of occurrences of `count_value` in `arr_a`.
3. The array after adding the `broadcast_value` to each element.
4. The sorted array.

Code:

```
import numpy as np

# Input array from the user
arr_a = np.array(list(map(int, input().split()))))

# Searching
search_value = int(input('Value to search: '))
count_value = int(input('Value to count: '))
broadcast_value = int(input('Value to add: '))

# Find indices where value exists in array
arr_a.where(arr_a[i] == search_value)[4]

# Count occurrences in array
num_count = np.count_nonzero(arr_a == count_value)
print(r)
```

```
# Broadcasting: add type
dummy = broadcast_value

# Sort the first array
arr_a.sort(arr_a)
print(B)
```

Output:

Average time: **0.022 s**
22.25 ms

Maximum time: **0.033 s**
33.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 53 ms Debug

Expected output

1 1 1 2 2 2

Value to search: 1

Value to count: 2

Value to add: 2

[0 1 2]

3

[3 3 3 4 4 4]

[1 1 1 2 2 2]

Actual output

1 1 1 2 2 2

Value to search: 1

Value to count: 2

Value to add: 2

[0 1 2]

3

[3 3 3 4 4 4]

[1 1 1 2 2 2]

Test case 2 18 ms Debug

Expected output

1 2 3 4 5

Value to search: 6

Value to count: 6

Value to add: 6

[]

0

[7 8 9 10 11]

[1 2 3 4 5]

Actual output

1 2 3 4 5

Value to search: 6

Value to count: 6

Value to add: 6

[]

0

[7 8 9 10 11]

[1 2 3 4 5]

3.2.7. Student Data Analysis using NumPy

Problem Statement:

Write a Python program that uses the data in a 2D NumPy array (including roll numbers and subject marks) to perform the following operations:

- Find total students: Display the number of records in the dataset.
- Find all students: Display all student roll numbers and their marks.
- Find all subjects for a student: Given a roll number, display all the marks of all subjects for that student.
- Find maximum marks in Subject: Find the maximum marks of all subjects and list the marks.
- Find minimum marks of each subject: Find the minimum mark for each subject.
- Find total marks of each student: Calculate the total marks for each student and list the totals.
- Find average marks of S1 and S2: Compute the average of marks for Subject 1 and Subject 2.
- Find average marks of S1 and S3: Compute the average of marks for Subject 1 and Subject 3.
- Find the average marks of each subject: Find the average marks of each subject and display the result.
- Find roll no who got maximum marks in Subject: Count the number of students who got marks in Subject 1 < 40.
- Find the count of students who got marks in Subject 2 > 90.
- Find the count of students in each subject who got marks >= 90.
- Find the count of subjects in which student scored >= 90: For each student, find the count of subjects in which they scored >= 90.
- Find the name of students who got less than 40 marks in subject: Find the number of students who got less than 40 marks in subject 1 (i.e., marks < 40).
- Find most qualified: Find the top marks qualifiers among all students who passed in Subject 3.

Note: Use the training data to perform the above mentioned operations.

Code:

```
import numpy as np

# Loading student data (2D array: roll no + subject marks)
student_data = np.array([
    [301, 67, 77, 88],
    [302, 78, 88, 77],
    [303, 45, 56, 89],
    [304, 88, 98, 45],
    [305, 78, 88, 95],
    [306, 68, 78, 36],
    [307, 79, 12, 45],
    [308, 46, 45, 77],
    [309, 89, 32, 88],
    [310, 79, 24, 33]
])

# 1
print(f'Total Students: {len(student_data)}')
```

```

# 2
print('All Student Details:')
print(student_data)

# 3
print(f'Total Students at half {student_data[:, 0]}, by.sum({student_data[:, 1]},
{student_data[:, 1]}, {student_data[:, 1]}, sum=48)')
# ... (additional operations continue)
print(f'Subject 1 Marks {student_data[:, 1]}')
print(f'Min marks in Subject 2 {student_data[:, 2].min()}')
print(f'Max marks in Subject 3 {student_data[:, 3].max()}')
print(f'All subject marks: {student_data[:, 1:]}')
print(f'Average Marks(mean):', np.mean(student_data[:, 1:], 1))
print(f'Average marks of each subject:', np.mean(student_data[:, 1:], 0))
print(f'Average Marks of S1 and S2 {np.mean(student_data[:, [1, 2]], axis=0)}')
print(f'Average Marks of S1 and S3 {np.mean(student_data[:, [1, 3]], axis=0)}')
print(f'Total Marks {np.sum(student_data[:, 1:], axis=1)}')
print(f'Count of students who got marks in Subject 1 < 40 {(student_data[:, 1] <
40).sum()}')
print(f'Count of students who got marks in Subject 2 > 90: {(student_data[:, 2] >
90).sum()}')
# ... remaining operations
print(student_data[j, :])

```

Output:

Explorer

Average time:0.035 s35.00 ms

Maximum time:0.035 s35.00 ms

1 out of 1 shown test case(s) passed

Debugger

Test case 10.035 msDebug

Expected output

Actual output

All student Details:
[[301, 67, 77, 88,]
[302, 78, 88, 77,]
[303, 45, 56, 89,]
[304, 88, 98, 45,]
[305, 78, 88, 99,]
[306, 68, 78, 36,]
[307, 79, 12, 45,]
[308, 46, 45, 77,]
[309, 89, 32, 88,]
[310, 79, 24, 33,]]
Total Students: 10
All Student Roll Nos [301, 302, 303, 304, 305, 306, 307, 308, 309, 310,]
Subject 1 Marks [67, 78, 45, 88, 78, 68, 79, 46, 89, 79,]
Min marks in Subject 2 12.0
Max marks in Subject 3 99.0
All subject marks: [[67, 77, 88,]
[78, 88, 77,]
[45, 56, 89,]
[88, 98, 45,]
[78, 88, 99,]
[68, 78, 36,]
[79, 12, 45,]
[46, 45, 77,]
[89, 32, 88,]
[79, 24, 33,]]
Total Marks [232, 243, 190, 231, 265, 182, 136, 168, 209, 136,]
[77.3 81, 63.3 77, 88.3 60.7 45.3 56, 69.7 45.3]
Average Marks of each subject [71.7 59.8 67.7]
Average Marks of S1 and S2 [71.7 59.8]
Average Marks of S1 and S3 [71.7 67.7]
Roll no who got maximum marks in Subject 3 305.0
Roll no who got minimum marks in Subject 2 307.0
Roll no who got 24 marks in Subject 2 [[310,]]
Count of students who got marks in Subject 1 < 40 0
Count of students who got marks in Subject 2 > 90: 1
Count of students in each subject who got marks >= 90: [0 1 1]
Roll no: [301, 302, 303, 304, 305, 306, 307, 308, 309, 310,]
Count of subjects in which student got marks >= 90: [0 0 0 1 1 0 0 0 0 0]
[45, 46, 67, 68, 78, 78, 79, 79, 88, 89,]
[[301, 67, 77, 88,]
[302, 78, 88, 77,]
[304, 88, 98, 45,]
[305, 78, 88, 99,]
[306, 68, 78, 36,]
[307, 79, 12, 45,]
[309, 89, 32, 88,]
[310, 79, 24, 33,]]

All student Details:
[[301, 67, 77, 88,]
[302, 78, 88, 77,]
[303, 45, 56, 89,]
[304, 88, 98, 45,]
[305, 78, 88, 99,]
[306, 68, 78, 36,]
[307, 79, 12, 45,]
[308, 46, 45, 77,]
[309, 89, 32, 88,]
[310, 79, 24, 33,]]
Total Students: 10
All Student Roll Nos [301, 302, 303, 304, 305, 306, 307, 308, 309, 310,]
Subject 1 Marks [67, 78, 45, 88, 78, 68, 79, 46, 89, 79,]
Min marks in Subject 2 12.0
Max marks in Subject 3 99.0
All subject marks: [[67, 77, 88,]
[78, 88, 77,]
[45, 56, 89,]
[88, 98, 45,]
[78, 88, 99,]
[68, 78, 36,]
[79, 12, 45,]
[46, 45, 77,]
[89, 32, 88,]
[79, 24, 33,]]
Total Marks [232, 243, 190, 231, 265, 182, 136, 168, 209, 136,]
[77.3 81, 63.3 77, 88.3 60.7 45.3 56, 69.7 45.3]
Average Marks of each subject [71.7 59.8 67.7]
Average Marks of S1 and S2 [71.7 59.8]
Average Marks of S1 and S3 [71.7 67.7]
Roll no who got maximum marks in Subject 3 305.0
Roll no who got minimum marks in Subject 2 307.0
Roll no who got 24 marks in Subject 2 [[310,]]
Count of students who got marks in Subject 1 < 40 0
Count of students who got marks in Subject 2 > 90: 1
Count of students in each subject who got marks >= 90: [0 1 1]
Roll no: [301, 302, 303, 304, 305, 306, 307, 308, 309, 310,]
Count of subjects in which student got marks >= 90: [0 0 0 1 1 0 0 0 0 0]
[45, 46, 67, 68, 78, 78, 79, 79, 88, 89,]
[[301, 67, 77, 88,]
[302, 78, 88, 77,]
[304, 88, 98, 45,]
[305, 78, 88, 99,]
[306, 68, 78, 36,]
[307, 79, 12, 45,]
[309, 89, 32, 88,]
[310, 79, 24, 33,]]

< Prev

Reset

Submit

Next >